Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Algorithmic Trading and Market Simulation Platform

Team Name: ***Team ID:***	*TradeSim* *DSCPP-III-2026-T331*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Jatrana, Satvik – 2510019213* *2510019213@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Bhatt, Siddharth – 2510012106* *2510012106@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Uniyal, Akshat – 2510370489* *2510370489@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Kumar, Sourabh – Student ID* *email ID* picture

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

TradeSim is an Algorithmic Trading and Market Simulation Platform developed using C and C++ which focuses on demonstrating the fundamental operations of a simulated stock market. The project aims to explain how the buy and sell orders are managed, work, and executed while demonstrating the practical application of Data Structures and Object-Oriented Programming (OOP) in a trading environment.

The platform combines the C data structures and algorithms, including arrays, linked lists, queues, searching, and sorting, with C++ concepts such as classes, objects, encapsulation, and abstraction. Its main features include virtual traders, an order book that organizes and manages orders according to price-time priority, and a matching engine that supports compatible buy and sell orders, including complete and partial fills. It also includes trade records, portfolio management, cash and holdings tracking, realized profit and loss (P&L) calculation, a menu-driven interface, and file handling to store order and trade records.

The expected outcome is a functional market simulation platform that demonstrates how data structures, algorithms, and object-oriented design can be applied to trading-system operations. The project is developed using GCC/G++, Visual Studio Code, and Git/GitHub. Although the current implementation focuses on simulated trading rather than a live stock exchange, it provides a foundation for future enhancements such as historical market data integration, trading strategies, and backtesting.

Updated Project Approach and Architecture

- **Development Approach and Architecture:** TradeSim is developed as an application using C and C++, where C is used for data structure modules, while C++ is used to build the main application using Object-Oriented Programming (OOP). The system follows this workflow: Virtual Trader $\rightarrow$ Order Management $\rightarrow$ Order Book $\rightarrow$ Matching Engine $\rightarrow$ Trade Records $\rightarrow$ Portfolio Manager.
- **Order Management and Matching:** Traders can place BUY and SELL orders by entering the price and quantity. The system validate orders and check the available cash or holdings. The order book arranges buy orders from highest to lowest price and sell orders from lowest to highest price. When the prices are equal, earlier orders receive priority. The matching engine attempts to execute trades when the best buy price is equal to or greater than the best sell price,which also supports complete and partial order execution.
- **Trade and Portfolio Management:** Executed trades are recorded, and portfolios track cash, holdings, total buy value, total sell value, and realized profit and loss (P&L).
- **Subject Integration and Tools:** C modules include linked lists, queues, and array utilities. C++ classes such as Order, Trader, Trade, Portfolio, OrderBook, and MatchingEngine manage the main components. The project uses the C++ Standard Library, file handling, GCC/G++, Visual Studio Code, and Git/GitHub.
- **File Handling and Testing:** The system can save order and trade records to files. Initial testing covers trader creation, order placement, input validation, portfolio display, and file saving. Further end to end testing is required to verify trade execution and portfolio updates.

Tasks Completed

Tasks Completed and Team Member :

1. Requirement analysis and system architecture design **[Satvik Jatrana]**
2. Implementation of C data structures, including linked lists, queues, and array utilities **[Siddharth Bhatt]**
3. Development of C++ classes: Order, Trader, Trade, and Portfolio**[Akshat Uniyal]**
4. Implementation of OrderBook with buy/sell order management and price-time priority **[Satvik Jatrana]**
5. Development of MatchingEngine with order-matching and partial-execution logic**[Satvik Jatrana]**
6. Implementation of portfolio management and basic realized P&L calculations**[Akshat Uniyal]**
7. Development of the menu-driven console interface for trader and order management**[Akshat Uniyal]**
8. Implementation of file handling to save order and trade records**[Siddharth Bhatt]**
9. Initial testing of trader creation, order placement, input validation, portfolio display, and file saving **[All three members]**

Challenges/Roadblocks

- **Order Matching:** Matching buy and sell orders while handling complete and partial execution requires careful updating of order quantities and statuses. This will be addressed through additional testing of different trading scenarios.
- **Portfolio Management:** Maintaining correct cash balances and holdings after trade execution is an important challenge. The system will be tested with two traders to verify that both portfolios are updated correctly.
- **Order Validation:** The system must prevent invalid orders and reject orders when traders have insufficient cash or holdings. Validation checks have been implemented and will be tested with more input cases.
- **Pending Order Management:** Multiple pending orders may use the same available cash or holdings. A resource reservation mechanism will be developed to prevent orders from exceeding available resources.
- **Integration of C and C++:** The C data structure modules and C++ trading components have been developed separately. Their integration and practical usage will be reviewed and documented to demonstrate the connection between both subjects.
- **File Handling:** Order and trade records can be saved, but additional work may be required to store portfolio information and restore saved data.
- **Testing and Debugging:** Initial functional checks have been performed, but complete trade execution, order cancellation, price-time priority, and boundary cases require further testing.

These challenges will be addressed through modular development, debugging, integration testing, and verification of the application's output before the final demonstration.

Tasks Pending & Team Member (to complete the task)

Task Pending:

- Test complete and partial trade execution between two virtual traders **[Satvik Jatrana]**
- Fix portfolio consistency and verify cash and holdings updates after trades **[Akshat Uniyal]**
- Implement order cancellation and test cancelled orders **[Satvik Jatrana]**
- Add reservation of cash and holdings for pending orders **[Akshat Uniyal]**
- Integrate and test C data structures with the main C++ application where appropriate **[Siddharth Bhatt]**
- Test price-time priority, invalid inputs, insufficient funds, and boundary cases **[All three members]**
- Improve file handling and evaluate saving portfolio information **[Siddharth Bhatt]**
- Complete system integration, update documentation, and prepare the final demonstration **[Satvik Jatrana (integration), Akshat Uniyal and Siddharth Bhatt (documentation and demonstration support)]**

Summary of Pending Work:

- **Trading validation:** Verify complete and partial trade execution and the full workflow from order placement to portfolio updates.
- **Portfolio and order management:** Resolve account consistency issues, implement order cancellation, and manage resources for pending orders.
- **Integration and testing:** Connect the C data structure modules where appropriate, improve file handling, and test normal, invalid, and boundary cases.
- **Final submission:** Prepare test results, screenshots, project documentation, and the final presentation with contributions from all team members.

Project Outcome/Deliverables

1. **TradeSim Application:** For virtual traders , a menu driven application known as TradeSim has been developed using C and C++ , to place buy and sell orders.
2. **Order Book and Matching Engine:** Separate books for buy and sell orders ,that has features like price-time priority and also supports both complete and partial order execution.
3. **Data Structures and Algorithms:** Demonstrate the application of data structures in order management by using linked lists, queues, arrays, searching, and sorting .
4. **Trade Records:** Recording and displaying the executed trades,order IDs, execution prices, and their quantities.
5. **Portfolio Management:** Tracking virtual cash, holdings, total buy value, total sell value, and basic realized profit and loss (P&L).
6. **File Handling:** Saving orders and trade records using file based storage.
7. **Testing and Validation:** Testing of the features like trader creation,order placement, portfolio display, and file saving , and overall working of the application.
8. **Source Code and Documentation:** A GitHub repository contains the project source code, project report, presentation, and the testing evidences.
9. **Learning Outcome and Future Scope:** Demonstration of the practical application of Data Structures in C and Object-Oriented Programming in C++ in this simulated trading application. Future enhancements may include historical market data, trading strategies, and backtesting.

Progress Overview

- **Overall Progress:** TradeSim has progressed from the initial proposal to a menu driven application with the trading features implemented.
- **Completed Work:** C++ classes, trader management, buy and sell order placement, order-book organization, price-time priority, matching logic, portfolio management, file handling, and the menu driven interface have been developed.
- **Testing Progress:** the tests cover trader creation, order placement, input validation, order-book display, portfolio display, and saving order and trade records.
- **Work in Progress:** Testing of complete and partial trade execution and portfolio updates , and Integration of the C data structure modules with the main C++ workflow also requires further review.
- **Outstanding Tasks:** Portfolio working, order cancellation,reservation of cash and holdings for pending orders, and final documentation remain to be completed.
- **Next Steps:** Resolve remaining issues,validate complete trading scenarios, review integration, document the test results, and prepare the final demonstration.

Codebase Information

Repository Link: https://github.com/NJR004/TradeSim
Programming Languages: C and C++.
Development Environment: Visual Studio Code with GCC/G++ compiler.
Version Control: Git and GitHub.
Application Type: Menu-driven console application.

Main Components:

1. **Order.h/.cpp —** Order creation, validation, status management, and automatic order IDs.
2. **Trader.h/.cpp —** Virtual trader information and account details.
3. **Trade.h/.cpp —** Records executed trades.
4. **Portfolio.h/.cpp —** Manages cash, holdings, transaction values, and realized P&L.
5. **OrderBook.h/.cpp —** Maintains buy and sell orders and applies price-time priority.
6. **MatchingEngine.h/.cpp —** Matches compatible orders and manages trade execution.
7. **FileManager.h/.cpp —** Saves order and trade records.
8. **C data-structure modules —** Implement linked-list operations, queue operations, and array utilities.
9. **main.cpp —** Provides the menu-driven interface and connects the main application components.

Testing and Validation Status

A. Completed Tests :

- Trader Creation Test : Successfully created a virtual trader and displayed trader details.
- Automatic Order ID : Test The system automatically generated Order ID 1000.
- BUY Order Placement Test : A valid BUY order was accepted and displayed in the order book.
- Input Validation Test : The system rejected a SELL order when the trader had insufficient holdings.
- Order Book Display Test : The pending BUY order appeared correctly in the buy order book.
- No-Match Scenario Test : No trade was executed when a compatible SELL order was unavailable.
- Portfolio Display Test : The portfolio displayed cash, holdings, transaction values, and P&L.
- File Handling Test : The application reported successful saving of order and trade records.

B. Pending Tests :

- Two-Trader Matching Test : Verify successful trade execution between two different traders.
- Partial Execution Test : Verify remaining order quantity and order status after partial execution.
- Portfolio Update Test : Verify cash and holdings updates for both traders after a trade.
- Order Cancellation Test : Test cancellation and removal of cancelled orders from the order book.
- Boundary and Integration Tests : Test price-time priority, insufficient funds, invalid inputs, boundary cases, and complete workflow integration.

C. Testing Summary :

- **Completed:** 8 tests passed.
- **Pending:** 5 tests require execution and verification.
- **Failed:** No failed tests have been recorded in the available test results.

The initial tests confirm that the basic trader management, order placement, order-book display, portfolio display, and file-saving functions work in the tested scenarios. Further testing is required to validate complete and partial trade execution, portfolio updates, order cancellation, and end-to-end system integration.

Deliverables Progress and Current Status

- **TradeSim Console Application :** Core features are implemented; end-to-end testing and remaining fixes are required.
- **Order Book and Matching Engine :** Order management, price-time priority, and matching logic are implemented; complete and partial execution require further validation.
- **Data Structures and Algorithms in C :** Linked lists, queues, and array utilities are implemented; integration and testing evidence remain outstanding.
- **Trade Records :** Trade record creation and display are implemented; successful execution scenarios need further verification.
- **Portfolio Management and P&L :** Cash and holdings updates and basic realized P&L calculations are implemented; portfolio consistency requires further testing.
- **File-Based Storage :** Saving order and trade records has been tested; portfolio storage and loading saved data remain outstanding.
- **Test Cases and Results :** Initial functional tests have passed; comprehensive matching, partial-fill, cancellation, and boundary tests remain.
- **Source Code and GitHub Repository :** The repository is available; final code, documentation, and commit details need verification.
- **Project Report and Presentation :** The Phase-II report is being prepared; final testing evidence, screenshots, and presentation materials remain to be completed.
- **Historical Market Data, Trading Strategies, and Backtesting :** These are proposed future extensions and are not part of the current core implementation.